Servizi del SO

I **sistemi operativi** forniscono l'**ambiente in cui si eseguono i programmi**.
E' quindi necessario definire i servizi che ogni SO deve offrire:

- **Comunicazioni:** scambio di informazioni tra processi in esecuzione. Può avvenire tramite memoria condivisa o scambio di messaggi.
- **Rilevamento errori:** rilevare errori nella CPU, nella memoria, nei dispositivi di I/O.
- **Allocazione di risorse:** gestire le risorse in modo da poter eseguire quanti più processi possibile in contemporanea.
- **Contabilizzazione delle risorse:** il SO deve sapere quante e quali risorse sono allocate ai suoi processi.
- **Protezione e sicurezza:** garantire la protezione delle informazioni in memoria da accessi indesiderati e processi concorrenti.

CLI

Si tratta dell'interfaccia utente a **riga di comando**. Permette di impartire comandi al SO.
Può essere implementata nel kernel o tramite programmi a parte (UNIX/Linux).
La sua funzione è quella di interpretare ed eseguire le istruzioni fornite tramite un **interprete dei comandi**.

I comandi sono di due tipologie:

- **Comandi dell'interprete:** sono interni al codice dell'interprete stesso. Invocandoli, si effettua un salto all'opportuna sezione di codice. Il numero di comandi interni determina la dimensione dell'interprete.
- **Comandi esterni:** possono essere aggiunti dal programmatore posizionando file con nomi appropriati all'interno di alcune directory specifiche. Non essendo parte del codice dell'interprete, non ne modificano le dimensioni.

In molti sistemi, la **maggior parte delle funzioni** è disponibile **solo tramite CLI**.
L'interfaccia CLI semplifica l'esecuzione di comandi ripetuti, in quanto sono programmabili.
E' possibile creare **script** composti da **più comandi consecutivi**, che vengono interpretati a riga di comando.



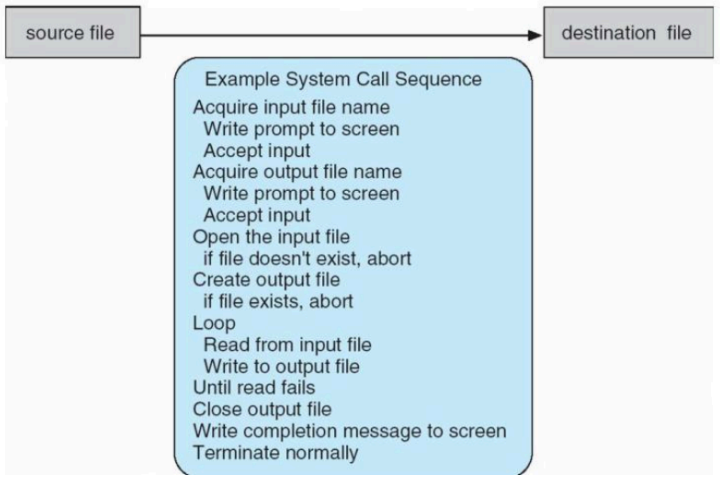


GUI

Interfaccia più **user friendly** usata per realizzare il **desktop**.
L'**interazione** risulta più **semplice** tramite mouse e tastiera.
Le **icone** rappresentano programmi, file, cartelle, etc...
I diversi tasti del mouse rappresentano specifiche azioni.
Questa tipologia di interfaccia nasce alla fine degli anni '70.

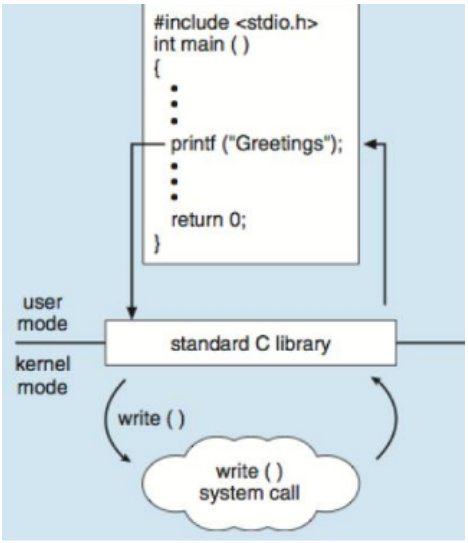
Chiamate di sistema

Forniscono l'interfaccia tra i processi e i servizi offerti dal SO.
Sono realizzate in **linguaggi di alto livello** come C o C++.
Vengono generalmente invocate tramite **API** (Application Programming Interface), come la WIN64 API e la POSIX API.



⇨Migliaia di chiamate di sistema al secondo!

A ciascuna system call è **associato un numero**.
L'**interfaccia alle chiamate a sistema** mantiene una **tabella** con tutti i numeri, **effettua la chiamata** e **restituisce** lo **stato del sistema** dopo l'esecuzione, oltre agli eventuali **valori di ritorno**.



Un esempio è la chiamata alla funzione **printf()** in C.

Questa invoca la syscall **write()**.

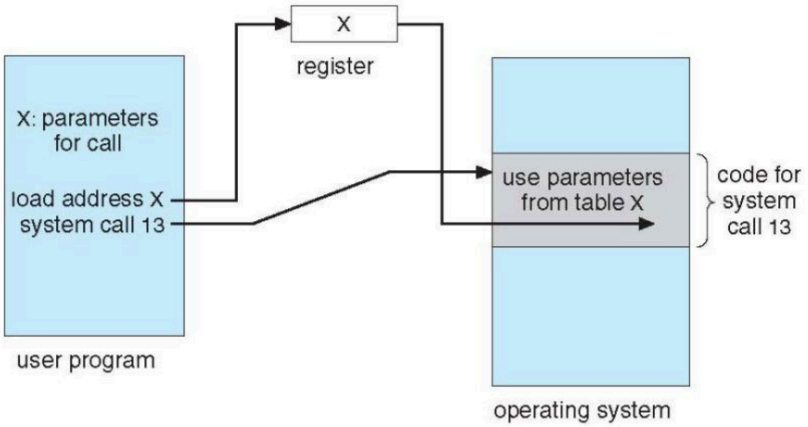
La **libreria standard** riceve dal sistema l'output della chiamata a sistema e lo passa al programma utente.

Passaggio dei parametri

L'**informazione necessaria** per la chiamata a sistema spesso non richiede solo il nome della chiamata, ma anche alcuni **parametri**.
Chiamate diverse possono necessitare di tipi e quantità diverse di parametri.

Il passaggio di parametri avviene in 3 modalità:

- **Registri:** vengono passati i parametri tramite i registri della CPU.
  E' rapido, ma talvolta sono necessari più parametri di quanti siano i registri.
- **Memorizzazione:** vengono salvati i parametri in un blocco di memoria e viene fornito alla syscall l'indirizzo del blocco. (Linux usa questo metodo per >5 parametri).
- **Push:** Si effettua il push dei parametri nello stack del programma. Il SO li recupera tramite un pop.



Tipi di chiamate a sistema

Una categoria di system call è quella legata alla **gestione di processi**:

- Creazione e terminazione (fork, exit).
- Caricamento ed esecuzione (exec, execve).
- Lettura, modifica degli attributi del processo (get/set attributes).
- Attesa di tempo o di un evento (wait, waitpid).
- Assegnazione e rilascio di memoria (alloc, free).
- Invio di segnali (signal, kill)

Lista di tutte le **altre tipologie**:

- Gestione delle informazioni
  - Lettura/modifica dell'ora e della data (`time`, `date`)
  - Informazioni sul sistema
  - Lettura/modifica degli attributi di processi, file e dispositivi (`ps`, `getpid`)
- Comunicazione
  - Apertura e chiusura di una connessione (open connection, close connection, `pipe`)
  - Invio e ricezione di messaggi (send, receive)
  - Informazioni sullo stato dei trasferimenti
  - Inserimento ed esclusione di dispositivi remoti
  - Condivisione della memoria (`shm_open`, `shmget`, `mmap`)
- Protezione
  - Controllo di accesso alle risorse
  - Lettura/modifica dei permessi di accesso (`chown`, `chmod`)
- Gestione dei file
  - Creazione e cancellazione di file (create, delete)
  - Apertura e chiusura di file (`open`, `close`)
  - Lettura, scrittura e posizionamento (`read`, `write`, seek)
  - Lettura/modifica degli attributi di un file (nome, tipo, codici di protezione, informazioni di contabilizzazione – get/set file attributes)
- Gestione dei dispositivi di I/O
  - Richiesta e rilascio di un dispositivo (request, release)
  - Lettura, scrittura e posizionamento (`read`, `write`, seek)
  - Lettura/modifica degli attributi di un dispositivo (`ioctl`)

Programmi di sistema

Esistono vari **programmi forniti dal SO** per lo sviluppo e l'esecuzione di programmi utente.

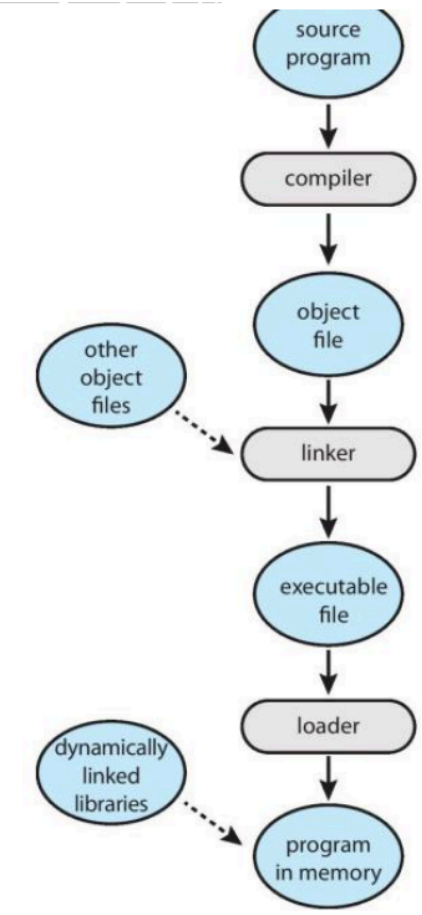
- **Gestione file**: (cp, rm, ...)
- **Informazioni di stato**: (du, who, whoami, ...)
- **Editing di file**: (vi)
- **Supporto a linguaggi di programmazione** (gcc)
- **Caricamento ed esecuzione di programmi**.
- **Comunicazioni**.

Il SO implementa anche alcuni **servizi di background**.
Alcuni di essi sono eseguiti in fase di boot, mentre altri sono sempre attivi da boot a shutdown.
Questi servizi si occupano di controllare il disco, eseguire lo scheduling dei processi, fare logging degli errori, gestire le connessioni di rete, etc...

Esistono poi i **programmi applicativi**, che non fanno parte del SO, sono eseguiti dagli utenti tramite CLI o GUI.

Linker e loader

Il codice sorgente viene compilato in **file oggetto rilocabili**, che possono essere caricati in qualsiasi posizione di memoria.
Questi file vengono combinati dal **linker** in un singolo **file eseguibile binario**, comprensivo anche delle funzioni di libreria.
Questi eseguibili risiedono in memoria secondaria fino alla loro esecuzione.
A quel punto il **loader** li carica in memoria primaria per eseguirli.



I moderni sistemi operativi **non collegano le librerie** ai file eseguibili.
Esistono infatti le **librerie collegate dinamicamente** (DLL in Windows), che vengono caricate quando necessarie e **condivise** da ogni programma che le utilizzi.

I file oggetto e gli **eseguibili** devono avere un **formato standard** in ogni SO, in modo che il sistema sappia come caricarli e avviarli.
UNIX utilizza **ELF**, Windows utilizza **PE**, MacOS utilizza **Mach-O**.

Gli unici modi per avere file eseguibili **compatibili con diversi SO** sono

- Utilizzare un linguaggio interpretato, come Python.
- Utilizzare un linguaggio che include una VM contenente l'applicazione, come Java.
- Ricompilare il sorgente per ogni diverso sistema in cui si vuole eseguire il programma.

Le **ABI** (Application Binary Interface) definiscono il modo in cui i componenti del codice binario possono interfacciarsi per un dato SO su una determinata architettura. Definisce come un'applicazione debba comunicare con se stessa, con il kernel, con le librerie, etc...

Progettazione di un SO

Quando si progetta un sistema operativo è necessario considerare i concetti di

- **Politiche:** quali sono i compiti e i servizi del sistema?
- **Meccanismi:** come sono realizzati questi compiti e servizi?

Sistemi come Windows o MacOs fissano a priori sia le politiche che i meccanismi. Le applicazioni hanno quindi interfacce simili in quanto l'interfaccia stessa fa parte del SO.
UNIX/Linux considera invece i due aspetti come nettamente separati.

Sviluppo di un SO

Storicamente, i SO venivano programmati in linguaggio **assembly**.
Successivamente furono creati linguaggi specifici per la programmazione di sistema, come **Algol** e **PL/1**.
Attualmente si usano anche linguaggi di alto livello come **C** o **C++**.

Generalmente, la suddivisione dei linguaggi è questa

- **Assembly** per le componenti di basso livello (driver).
- **C** per il kernel.
- **C, C++** per programmi di sistema.

Questa divisione rende lo sviluppo più rapido, il codice più compatto e leggibile, facile da mantenere e più portatile.

Kernel monolitici

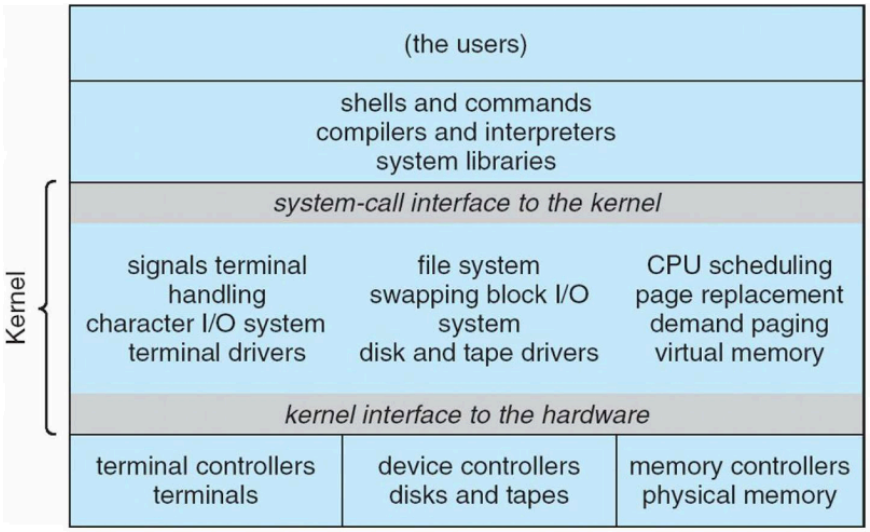
Le funzioni di gestione delle risorse sono realizzate dentro il kernel.
L'**intero SO** tende a identificarsi col **kernel**.
Avere tutte le funzioni all'interno dello stesso spazio di indirizzi può provocare il **blocco dell'intero sistema** in caso di **bug** in uno dei moduli.

I kernel monolitici sono sistemi generalmente **storici**. Oggi si cerca di mantenere separati i vari moduli del SO.

Esempi di sistemi monolitici sono

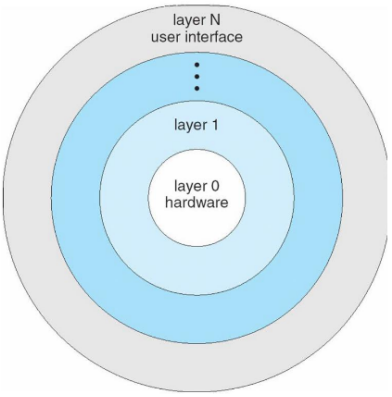
- **MS-DOS:** nessuna suddivisione in moduli. Le applicazioni accedono direttamente alle routine di sistema. Nessuna protezione da eventuali attacchi dei programmi utenti.
- **UNIX:** struttura scarsamente stratificata data dalle limitazioni hardware dell'epoca. Nel kernel c'è tutto ciò che sta sopra l'hardware e sotto l'interfaccia delle chiamate a sistema. Tutte le funzioni, dallo scheduling alla gestione della memoria, risiedono nel kernel allo stesso livello.

La struttura di UNIX:



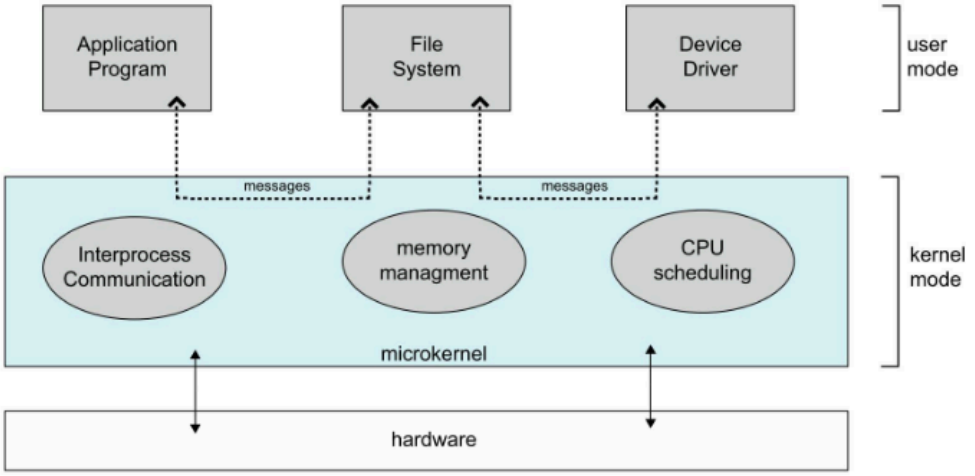
Approccio stratificato

Questa soluzione porta con sé alcuni **vantaggi**, come la **semplicità di sviluppo** , e degli **svantaggi**, come la **difficoltà nel definire gli strati**.
Infatti, ogni strato può sfruttare solamente le funzioni degli strati più interni rispetto a sé.
Inoltre, **ne risente molto l'efficienza** del sistema. Ogni strato comunica con quello inferiore tramite una chiamata, quello inferiore a sua volta scende di un livello con un'altra chiamata e così via.



Microkernel

La **maggior parte delle funzionalità** sono spostate a **livello utente**.
Le uniche funzioni del **kernel** prevedono la gestione dei **processi**, della **memoria** e della **comunicazione** tra moduli.



I **vantaggi** di questa struttura comprendono la **facilità di espansione dei servizi** (che sono sostanzialmente dei normali programmi utente), la **facilità di modifica del kernel**, la **portabilità** e la maggiore **affidabilità** (solo una minima parte del codice viene eseguito in modalità kernel).

Lo **svantaggio** principale di questa soluzione è l'**overhead di comunicazione** tra lato utente e lato kernel.

Kernel modulari

La maggior parte dei **SO moderni** implementa kernel modulari, che fanno uso di LKM (Loadable Kernel Module).
Ciascuno dei **moduli** implementa una **componente base** del kernel, con la propria interfaccia e le proprie funzioni.
Un'**interfaccia comune** permette la **comunicazione** tra moduli.
I moduli possono essere **caricati in memoria a seconda delle esigenze**, sia all'avvio del sistema che a runtime.

Questa architettura ricorda quella a strati, ma risolve il problema della scarsa flessibilità in quanto ogni modulo può comunicare con un altro, pur rimanendo separati.

Sistemi Ibridi

Molti sistemi moderni non si limitano a implementare una delle architetture descritte sopra, ma combinano diversi approcci.

Il kernel **Linux** è per la maggior parte un kernel monolitico, in modo da sfruttare il miglioramento delle prestazioni derivante dall'avere tutte le funzioni nello stesso spazio di indirizzi e a livello kernel.
Nonostante ciò, supporta l'aggiunta di nuovi moduli, anche a runtime.

Windows ha anch'esso un approccio per lo più monolitico, ma prevede alcune funzionalità tipiche dei microkernel, come il supporto a sottosistemi eseguiti in modalità utente.

MacOS e iOS

La struttura alla base di questi due SO è simile, nonostante siano progettati per dispositivi diversi.

- **Strato dell'interfaccia utente:** lo strato con cui l'utente interagisce direttamente: tramite mouse e tastiera per MacOS, tramite touch screen per iOS.
- **Strato degli ambienti applicativi:** forniscono le API per linguaggi di programmazione Objective-C e Swift.
- **Ambienti di base:** supportano grafica e contenuti multimediali.
- **Ambiente kernel:** Darwin, che include il microkernel Mach + kernel BSD UNIX.

MacOS è compilato per architetture **x86** (Intel) o **ARM** (serie M), mentre **iOS** per architetture **ARM**.
iOS è molto **più restrittivo** nei confronti degli sviluppatori.

Il microkernel **Mach** gestisce **memoria**, chiamate di procedura remota (**RPC**), comunicazione tra processi (**IPC**) e **scheduling**.
Il kernel **BSD** fornisce la **CLI**, i servizi legati al **file system** e alla **rete** e la API POSIX.

E' previsto il **supporto** per **moduli** caricabili dinamicamente, detti **estensioni del kernel**.

Android

Sistema sviluppato da **Open Handset Alliance** (Google + molte aziende mobile).
Costituito da una serie di strati software. Il sistema è **open-source**.
Alla **base** del SO c'è una versione modificata del **kernel Linux**.

L'ambiente runtime comprende le librerie di base e la macchina virtuale **ART** (**Android Run Time**).

Le applicazioni sono programmate in Java sfruttando l'Android APi.

Il sistema, essendo utilizzabile da una grandissima gamma di dispositivi, include un **hardware abstraction layer** (**HAL**).

Debugging

Il **debugging** è l'attività di **individuazione e risoluzione di errori di sistema** (bug).

Il SO può generare **file di log** che danno informazioni sugli errori rilevati durante l'esecuzione di un processo.
E' anche in grado di acquisire e memorizzare in un file il contenuto della memoria utilizzata da un processo (**core dump**).
Tale file può essere fornito ad un programma di supporto (**debugger**) in grado di individuare l'errore.

Un **guasto del kernel** si definisce **crash**.
Anche in questo caso, è possibile salvare file log e core dump.

Tuning

Anche i problemi legati alle prestazioni sono considerati bug.
Dunque il **performance tuning** rientra tra le pratiche di debug.

Si tratta di un insieme di tecniche atte ad **ottimizzare le prestazioni** del sistema, **riducendo i bottleneck**.
Si esegue il codice e si analizza il suo comportamento osservando i file di log, in modo da individuare ostacoli e inefficienze.

All'interno del SO sono spesso presenti strumenti di monitoraggio che permettono all'amministratore o agli utenti di controllare l'uso delle risorse.
La tecnica del **profiling** prevede un campionamento periodico dell'instruction pointer, in modo da individuare le chiamate a sistema maggiormente usate.

Sottosistema Windows per Linux

L'architettura ibrida consente l'esecuzione di sottosistemi in grado di emulare altri SO.

Il **WSL** (Windows Subsystem for Linux) consente ad applicazioni linux-native di essere eseguite su Windows.
Ogni chiamata a sistema in Linux deve essere tradotta in una chiamata Windows tramite i servizi kernel LXCore e LXSS.

